Frontend Modernization — Plan 1 of 3 — Foundation

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Land the design-token system, font loading, redesigned UI primitives, and the brand + operator shells — all of which the per-page reflow (Plan 2) consumes.

Architecture: Three concentric layers — tokens (one CSS file + Tailwind shim) → primitives (~12 components) → shells (BrandShell + OperatorShell + Sidebar + Topbar + RightRail). Variables scoped by `[data-surface="brand"|"operator"][data-theme="dark"|"light"]` on `<body>`, set by the route shell on mount. Replaces existing `src/components/ui/{Button,Card,Spinner}.tsx` and the legacy Tailwind palette outright.

Tech Stack: Vite + React 18 + TypeScript + Tailwind CSS, Framer Motion 12 (already installed), `@fontsource-variable/mona-sans`, `@fontsource-variable/jetbrains-mono`, `@fontsource/dm-serif-display`, `@axe-core/playwright`, Vitest, Playwright.

Spec: `docs/superpowers/specs/2026-04-28-frontend-modernization-design.md` (commit `4ecd32a`)

TDD discipline (hybrid, per spec): - `[TDD]` — token resolution, primitive props/states, axe-core a11y, reduced-motion behavior, focus management. - `[Visual]` — purely visual reflows verified via the route-sweep diff. - `[Both]` — shells, where TDD covers integration and visual-diff covers appearance.

File Structure

Created in this plan

```
docs/superpowers/plans/2026-04-28-frontend-modernization-plan-1-foundatior
src/frontend/src/styles/tokens.css
src/frontend/src/styles/motion.css
src/frontend/src/components/primitives/Surface.tsx
src/frontend/src/components/primitives/Button.tsx
src/frontend/src/components/primitives/Card.tsx
src/frontend/src/components/primitives/Input.tsx
src/frontend/src/components/primitives/Badge.tsx
src/frontend/src/components/primitives/Tabs.tsx
src/frontend/src/components/primitives/Dialog.tsx
src/frontend/src/components/primitives/Skeleton.tsx
src/frontend/src/components/primitives/Toast.tsx
src/frontend/src/components/primitives/Table.tsx
src/frontend/src/components/primitives/GradientField.tsx
src/frontend/src/components/primitives/index.ts
src/frontend/src/components/layout/BrandShell.tsx
src/frontend/src/components/layout/OperatorShell.tsx
src/frontend/src/components/layout/Sidebar.tsx
src/frontend/src/components/layout/Topbar.tsx
src/frontend/src/components/layout/RightRail.tsx
src/frontend/src/components/layout/index.ts
src/frontend/src/components/primitives/__tests__/Button.test.tsx
src/frontend/src/components/primitives/__tests__/Card.test.tsx
src/frontend/src/components/primitives/__tests__/Input.test.tsx
src/frontend/src/components/primitives/__tests__/Badge.test.tsx
src/frontend/src/components/primitives/__tests__/Tabs.test.tsx
src/frontend/src/components/primitives/__tests__/Dialog.test.tsx
src/frontend/src/components/primitives/__tests__/Tokens.test.tsx
src/frontend/src/components/layout/__tests__/BrandShell.test.tsx
src/frontend/src/components/layout/__tests__/OperatorShell.test.tsx
tests/visual/route-sweep.mjs
tests/visual/__baseline__/*.png
tests/visual/manifest.json
```

Modified in this plan

```
src/frontend/tailwind.config.js
src/frontend/src/index.css
src/frontend/index.html
src/frontend/src/main.tsx
src/frontend/src/App.tsx
src/frontend/package.json
src/frontend/package-lock.json
playwright.config.ts
```

Deleted in this plan

```
src/frontend/src/components/ui/Button.tsx
src/frontend/src/components/ui/Card.tsx
src/frontend/src/components/ui/Spinner.tsx
src/frontend/src/components/ui/index.ts
```

(Existing imports of these components are redirected to `components/primitives` in Phase D's compatibility task.)

Untouched in this plan

```
src/components/ui-enhanced/*
All page files in src/frontend/src/pages/*
Backend, AI service, Netlify functions, Supabase migrations
```

Phase A — Worktree + Visual Baseline (commit 1)

Task A1: Create the redesign worktree

Files: none yet — git operation only.

- [ ] **Step 1: Create the worktree off main**

```
git -C /Users/morlock/fun/02_PowerShell_Projects/psscript worktree add \
  ../psscript-redesign-2026-04-28 \
  -b redesign/2026-04-28-frontend-modernization
```

Expected output: `Preparing worktree (new branch 'redesign/2026-04-28-frontend-modernization')` followed by checkout summary.

- [ ] **Step 2: Verify the worktree**

```
ls /Users/morlock/fun/02_PowerShell_Projects/psscript-redesign-2026-04-28/
git -C /Users/morlock/fun/02_PowerShell_Projects/psscript-redesign-2026-04
```

Expected output: `gitdir: ...worktrees/psscript-redesign-2026-04-28` and the branch name.

- [ ] **Step 3: Switch working directory for all subsequent tasks**

```
cd /Users/morlock/fun/02_PowerShell_Projects/psscript-redesign-2026-04-28
```

All subsequent file paths in this plan are **relative to this worktree** unless prefixed with `/Users/`.

Task A2: Install visual-diff dependencies

Files: `src/frontend/package.json`, `package-lock.json` (modified)

- [ ] **Step 1: Install pixelmatch + pngjs in the frontend workspace**

```
cd src/frontend
npm install --save-dev pixelmatch@^7 pngjs@^7
```

Expected output: `added 2 packages` (or `up to date` if Playwright already pulled them).

- [ ] **Step 2: Install @axe-core/playwright at the workspace root**

```
cd ../..
npm install --save-dev @axe-core/playwright@^4
```

Expected output: package added under devDependencies.

- [ ] **Step 3: Verify Playwright config exists and lists tests dir**

```
grep -E "testDir|testMatch" playwright.config.ts
```

Expected output: shows `testDir: 'tests/e2e'` (or similar). Note the value — Phase A's route-sweep harness goes in a *separate* directory (`tests/visual/`) so it doesn't run by default.

Task A3: Write the route inventory manifest

Files: - Create: `tests/visual/manifest.json`

- [ ] **Step 1: Create the manifest with all 36 routes**

Create `tests/visual/manifest.json`:

```
{
  "brand": [
    { "route": "/login",            "page": "Login" },
    { "route": "/register",         "page": "Register" },
    { "route": "/landing",          "page": "LandingPage" },
    { "route": "/auth/callback",    "page": "AuthCallback" },
    { "route": "/pending-approval", "page": "PendingApproval" },
    { "route": "/404",              "page": "NotFound" }
  ],
  "operator": [
    { "route": "/dashboard",                "page": "Dashboard" },
    { "route": "/scripts",                  "page": "ScriptManagement" },
    { "route": "/scripts/upload",           "page": "ScriptUpload" },
    { "route": "/scripts/editor",           "page": "ScriptEditor" },
    { "route": "/scripts/1",                "page": "ScriptDetail" },
    { "route": "/scripts/1/analysis",       "page": "ScriptAnalysis" },
    { "route": "/search",                   "page": "Search" },
    { "route": "/categories",               "page": "Categories" },
    { "route": "/manage-files",             "page": "ManageFiles" },
    { "route": "/documentation",            "page": "Documentation" },
    { "route": "/documentation/crawl",      "page": "DocumentationCrawl" }
    { "route": "/documentation/data",       "page": "CrawledData" },
    { "route": "/chat",                     "page": "ChatWithAI" },
    { "route": "/chat/simple",              "page": "SimpleChatWithAI" },
    { "route": "/chat/history",             "page": "ChatHistory" },
    { "route": "/agentic",                  "page": "AgenticAIPage" },
    { "route": "/agentic/orchestration",    "page": "AgentOrchestrationPag
    { "route": "/analytics",                "page": "Analytics" },
    { "route": "/profile",                  "page": "Profile" },
    { "route": "/settings",                 "page": "Settings" },
    { "route": "/settings/api",             "page": "ApiSettings" },
    { "route": "/settings/appearance",      "page": "AppearanceSettings" }
    { "route": "/settings/application",     "page": "ApplicationSettings"
    { "route": "/settings/categories",      "page": "CategoriesSettings" }
    { "route": "/settings/data-maintenance","page": "DataMaintenanceSettin
    { "route": "/settings/notifications",   "page": "NotificationSettings"
    { "route": "/settings/profile",         "page": "ProfileSettings" },
    { "route": "/settings/security",        "page": "SecuritySettings" },
    { "route": "/settings/users",           "page": "UserManagement" },
```

```
    { "route": "/ui-components",            "page": "UIComponentsDemo" }
  ],
  "criticalDetail": [
    { "route": "/scripts/1",          "page": "ScriptDetail",   "interacti
    { "route": "/scripts/1/analysis", "page": "ScriptAnalysis", "interacti
    { "route": "/dashboard",          "page": "Dashboard",      "interacti
    { "route": "/settings/users",     "page": "UserManagement", "interacti
  ]
}
```

(If any route does not currently exist in `App.tsx`, the harness logs and continues — it's expected that some pages may render under different paths.)

- [ ] **Step 2: Verify the JSON is valid**

```
node -e "JSON.parse(require('fs').readFileSync('tests/visual/manifest.jsor
```

Expected output: no output, exit code 0. Any parse error means the JSON is malformed.

Task A4: Write the route-sweep harness

Files: - Create: `tests/visual/route-sweep.mjs`

- [ ] **Step 1: Create the harness with baseline + diff modes**

Create `tests/visual/route-sweep.mjs`:

```
#!/usr/bin/env node
/* eslint-disable no-console */
import { chromium } from '@playwright/test';
import fs from 'node:fs/promises';
import path from 'node:path';
import { PNG } from 'pngjs';
import pixelmatch from 'pixelmatch';

const BASE_URL = process.env.SWEEP_BASE_URL ?? 'http://127.0.0.1:4173';
const ROOT = path.resolve(path.dirname(new URL(import.meta.url).pathname))
const MODE = process.argv.includes('--baseline') ? 'baseline' : 'diff';
const OUT_DIR = MODE === 'baseline' ? '__baseline__' : '__current__';
const DIFF_DIR = '__diff__';
const VIEWPORT = { width: 1440, height: 900 };
const FULL_PAGE = true;
const SETTLE_MS = 800;

async function loadManifest() {
  return JSON.parse(await fs.readFile(path.join(ROOT, 'manifest.json'), 'u
}

function captureName(page, surface, theme, motion) {
  return `${surface}__${theme}__${motion}__${page.replace(/[^a-zA-Z0-9_-]/
}

async function ensureDir(p) {
  await fs.mkdir(path.join(ROOT, p), { recursive: true });
}

async function captureRoute(browser, route, surface, theme, motion, name)
  const ctx = await browser.newContext({
    viewport: VIEWPORT,
    colorScheme: theme === 'dark' ? 'dark' : 'light',
    reducedMotion: motion === 'reduce' ? 'reduce' : 'no-preference',
  });
  const page = await ctx.newPage();
  try {
    await page.goto(`${BASE_URL}${route}`, { waitUntil: 'networkidle', tim
    await page.waitForTimeout(SETTLE_MS);
```

```
    const buf = await page.screenshot({ fullPage: FULL_PAGE, type: 'png' }
    await fs.writeFile(path.join(ROOT, OUT_DIR, name), buf);
    return { ok: true, name };
  } catch (err) {
    return { ok: false, name, error: err.message };
  } finally {
    await ctx.close();
  }
}

async function diffOne(name) {
  const baselinePath = path.join(ROOT, '__baseline__', name);
  const currentPath = path.join(ROOT, '__current__', name);
  const diffPath = path.join(ROOT, DIFF_DIR, name);
  let baseBuf, curBuf;
  try {
    [baseBuf, curBuf] = await Promise.all([
      fs.readFile(baselinePath),
      fs.readFile(currentPath),
    ]);
  } catch {
    return { name, status: 'missing' };
  }
  const base = PNG.sync.read(baseBuf);
  const cur = PNG.sync.read(curBuf);
  if (base.width !== cur.width || base.height !== cur.height) {
    return { name, status: 'size-changed', baseline: [base.width, base.hei
  }
  const diff = new PNG({ width: base.width, height: base.height });
  const mismatch = pixelmatch(base.data, cur.data, diff.data, base.width,
  await fs.writeFile(diffPath, PNG.sync.write(diff));
  return { name, status: 'diffed', mismatchPixels: mismatch };
}

async function run() {
  const manifest = await loadManifest();
  await ensureDir(OUT_DIR);
  if (MODE === 'diff') await ensureDir(DIFF_DIR);

  const browser = await chromium.launch();
```

```
  const results = [];

  // Brand: dark only, both motion preferences
  for (const item of manifest.brand) {
    for (const motion of ['default', 'reduce']) {
      const name = captureName(item.page, 'brand', 'dark', motion);
      results.push(await captureRoute(browser, item.route, 'brand', 'dark'
    }
  }
  // Operator: dark + light, both motion preferences
  for (const item of manifest.operator) {
    for (const theme of ['dark', 'light']) {
      for (const motion of ['default', 'reduce']) {
        const name = captureName(item.page, 'operator', theme, motion);
        results.push(await captureRoute(browser, item.route, 'operator', t
      }
    }
  }
  // Critical detail captures
  for (const item of manifest.criticalDetail) {
    const name = captureName(`${item.page}__${item.interaction}`, 'operato
    results.push(await captureRoute(browser, item.route, 'operator', 'dark
  }

  await browser.close();

  const failed = results.filter((r) => !r.ok);
  console.log(JSON.stringify({ mode: MODE, total: results.length, failed:

  if (MODE === 'diff') {
    const diffSummary = [];
    for (const item of results.filter((r) => r.ok)) {
      diffSummary.push(await diffOne(item.name));
    }
    const totalMismatch = diffSummary.reduce((sum, d) => sum + (d.mismatch
    console.log(JSON.stringify({ diffSummary: diffSummary.slice(0, 10), to
  }

  process.exit(failed.length > 0 ? 1 : 0);
}
```

```
run().catch((err) => {
  console.error(err);
  process.exit(2);
});
```

- [ ] **Step 2: Make it executable and dry-run with the manifest count**

```
chmod +x tests/visual/route-sweep.mjs
node -e "const m=require('./tests/visual/manifest.json'); console.log('bra
```

Expected output: `brand= 6 operator= 30 critical= 4`. (6×2 + 30×4 + 4 = 136 captures — slightly fewer than the 148 in the spec because brand routes don't have a light theme; spec total of 148 included an over-count. Plan 1 uses the corrected value of 136.)

Task A5: Capture the baseline (against current main, pre-redesign)

Files: `tests/visual/__baseline__/*.png` (binary artifacts, committed)

- [ ] **Step 1: Build the production frontend**

```
cd src/frontend
npm run build
```

Expected output: `vite v* building for production... ✓ built in *s`. Errors here are pre-existing and must be reported, not fixed in this task.

- [ ] **Step 2: Start the preview server in the background**

```
nohup npm run preview -- --port 4173 --host 127.0.0.1 > /tmp/preview-basel
echo $! > /tmp/preview-baseline.pid
sleep 4
curl -fsS http://127.0.0.1:4173 -o /dev/null && echo OK || echo FAILED
```

Expected output: `OK`. If FAILED, check `/tmp/preview-baseline.log`.

- [ ] **Step 3: Run the baseline sweep**

```
cd ../..
SWEEP_BASE_URL=http://127.0.0.1:4173 node tests/visual/route-sweep.mjs --b
```

Expected output: a JSON summary with `total: 136` and `failed: 0` or a small number of failures if specific routes 404 (those are noted, not blockers — the redesign will visit the same routes).

- [ ] **Step 4: Stop the preview server**

```
kill $(cat /tmp/preview-baseline.pid)
rm -f /tmp/preview-baseline.pid
```

- [ ] **Step 5: Verify baseline contents**

```
ls tests/visual/__baseline__ | wc -l
```

Expected output: a number close to 136 (any failed routes won't have files).

- [ ] **Step 6: Commit**

```
git add tests/visual/manifest.json tests/visual/route-sweep.mjs tests/visu
  src/frontend/package.json src/frontend/package-lock.json package.json pa
git commit -m "$(cat <<'EOF'
chore(redesign): capture visual baseline before tokens land

Adds tests/visual/route-sweep.mjs (Playwright-driven full-page screenshots
across 36 routes × surface × theme × motion-pref combos) and the manifest
that drives it. Baseline PNGs are committed as binary artifacts so the
post-redesign diff has something to subtract from. Pixelmatch + pngjs +
@axe-core/playwright installed; no production code touched.

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Expected output: commit summary with `tests/visual/...` and a few hundred PNG files added.

Phase B — Token System (commit 2)

Task B1: [TDD] Write the token-resolution test

Files: - Create: `src/frontend/src/components/primitives/__tests__/Tokens.test.tsx`

- [ ] **Step 1: Write the failing test**

```
// src/frontend/src/components/primitives/__tests__/Tokens.test.tsx
import { describe, it, expect, beforeEach, afterEach } from 'vitest';
import { render } from '@testing-library/react';

const cases: Array<{
  surface: 'brand' | 'operator';
  theme?: 'dark' | 'light';
  variable: string;
  expected: string;
}> = [
  { surface: 'brand',    variable: '--surface-base',  expected: 'rgb(14, 2
  { surface: 'brand',    variable: '--accent',         expected: 'rgb(252,
  { surface: 'brand',    variable: '--ink-primary',    expected: 'rgb(245,
  { surface: 'operator', theme: 'dark', variable: '--surface-base', expect
  { surface: 'operator', theme: 'dark', variable: '--accent',        expec
  { surface: 'operator', theme: 'light',variable: '--surface-base',  expec
  { surface: 'operator', theme: 'light',variable: '--accent',        expec
];

describe('design tokens', () => {
  beforeEach(() => {
    document.body.removeAttribute('data-surface');
    document.body.removeAttribute('data-theme');
  });
  afterEach(() => {
    document.body.removeAttribute('data-surface');
    document.body.removeAttribute('data-theme');
  });

  cases.forEach(({ surface, theme, variable, expected }) => {
    it(`${surface}${theme ? '/' + theme : ''} resolves ${variable} to ${ex
      document.body.setAttribute('data-surface', surface);
      if (theme) document.body.setAttribute('data-theme', theme);
      const el = document.createElement('div');
      el.style.color = `var(${variable})`;
      document.body.appendChild(el);
      const computed = getComputedStyle(el).color;
      el.remove();
      expect(computed).toBe(expected);
```

```
    });
  });
});
```

- [ ] **Step 2: Run the test and confirm it fails**

```
cd src/frontend
npm run test:run -- src/components/primitives/__tests__/Tokens.test.tsx
```

Expected output: 7 failures with messages like `Expected: "rgb(14, 21, 37)" Received: ""` (because `tokens.css` doesn't exist yet).

Task B2: [TDD] Create tokens.css with brand surface tokens

Files: - Create: `src/frontend/src/styles/tokens.css`

- [ ] **Step 1: Write the brand token block**

Create `src/frontend/src/styles/tokens.css` with the following content (full file, will be extended in B3 / B4):

```
/* ============================================================
   PSScript design tokens — single source of truth.
   Spec: docs/superpowers/specs/2026-04-28-frontend-modernization-design.m
   ============================================================ */

[data-surface="brand"] {
  --surface-base:    #0E1525;
  --surface-raised:  #131C30;
  --surface-overlay: #1A2440;
  --surface-glass:   rgba(20, 28, 50, 0.72);

  --ink-primary:     #F5F1E8;
  --ink-secondary:   #C9C4B6;
  --ink-tertiary:    #8B8576;
  --ink-muted:       #4D4A40;
  --ink-inverse:     #0E1525;

  --accent:          #FCA5A5;
  --accent-soft:     rgba(252, 165, 165, 0.16);
  --cool:            #5EEAD4;
  --warm:            #FCA5A5;
  --violet:          #C4B5FD;

  --signal-success:  #5EEAD4;
  --signal-warning:  #F5C97A;
  --signal-danger:   #F87B7B;
  --signal-info:     #C4B5FD;

  --ring-focus:      #FCA5A5;
  --shadow-near:     0 1px 2px rgba(0, 0, 0, 0.40), 0 2px 8px rgba(0, 0, 0
  --shadow-far:      0 8px 24px rgba(0, 0, 0, 0.48), 0 24px 64px rgba(0, 0
  --shadow-glow:     0 0 80px rgba(252, 165, 165, 0.18),
                     0 0 160px rgba(94, 234, 212, 0.10);

  --blur-glass:      12px;
  --radius-sm:       4px;
  --radius-md:       8px;
```

```
  --radius-lg:       14px;
  --radius-xl:       20px;
}
```

- [ ] **Step 2: Import tokens.css from index.css**

Open `src/frontend/src/index.css` and add the import as the **first non-Tailwind line** (Tailwind directives stay at top):

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@import './styles/tokens.css';

/* (existing rest of file follows here, untouched for now) */
```

- [ ] **Step 3: Re-run the brand subset of the token test**

```
npm run test:run -- src/components/primitives/__tests__/Tokens.test.tsx -t
```

Expected output: 3 brand cases pass. Operator cases still fail.

Task B3: [TDD] Add operator dark tokens

Files: `src/frontend/src/styles/tokens.css` (modified)

- [ ] **Step 1: Append the operator-dark block**

Append to `src/frontend/src/styles/tokens.css`:

```
[data-surface="operator"] {
  --surface-base:    #0E1116;
  --surface-raised:  #171B22;
  --surface-overlay: #1F242D;
  --surface-glass:   rgba(23, 27, 34, 0.66);

  --ink-primary:     #EAE6DD;
  --ink-secondary:   #B8B2A4;
  --ink-tertiary:    #7C7669;
  --ink-muted:       #4D4A42;
  --ink-inverse:     #0E1116;

  --accent:          #C8F25C;
  --accent-soft:     rgba(200, 242, 92, 0.14);
  --cool:            #5EEAD4;
  --warm:            #FCA5A5;
  --violet:          #C4B5FD;

  --signal-success:  #5EEAD4;
  --signal-warning:  #F5C97A;
  --signal-danger:   #F5A8A8;
  --signal-info:     #8FB4D9;

  --ring-focus:      #C8F25C;
  --shadow-near:     0 1px 2px rgba(0, 0, 0, 0.40), 0 2px 8px rgba(0, 0, 0
  --shadow-far:      0 8px 24px rgba(0, 0, 0, 0.50), 0 24px 64px rgba(0, 0
  --shadow-glow:     0 0 200px rgba(94, 234, 212, 0.04),
                     0 0 360px rgba(196, 181, 253, 0.04);

  --blur-glass:      12px;
  --radius-sm:       4px;
  --radius-md:       8px;
  --radius-lg:       14px;
  --radius-xl:       20px;
}
```

- [ ] **Step 2: Re-run the operator-dark subset**

```
npm run test:run -- src/components/primitives/__tests__/Tokens.test.tsx -t
```

Expected output: 2 operator-dark cases pass.

Task B4: [TDD] Add operator soft-light tokens

Files: `src/frontend/src/styles/tokens.css` (modified)

- [ ] **Step 1: Append the operator-light block**

Append to `src/frontend/src/styles/tokens.css`:

```
[data-surface="operator"][data-theme="light"] {
  --surface-base:    #FBF8F1;
  --surface-raised:  #FFFFFF;
  --surface-overlay: #FFFFFF;
  --surface-glass:   rgba(255, 255, 255, 0.78);

  --ink-primary:     #1A1F26;
  --ink-secondary:   #4D5663;
  --ink-tertiary:    #767F8C;
  --ink-muted:       #B5BBC4;
  --ink-inverse:     #FBF8F1;

  --accent:          #6B8F2E;
  --accent-soft:     rgba(107, 143, 46, 0.10);
  --cool:            #1F8B7A;
  --warm:            #C8623F;
  --violet:          #6B5DD3;

  --signal-success:  #1F8B7A;
  --signal-warning:  #B57B14;
  --signal-danger:   #C04646;
  --signal-info:     #335E8C;

  --ring-focus:      #6B8F2E;
  --shadow-near:     0 1px 2px rgba(0, 0, 0, 0.06), 0 2px 8px rgba(0, 0, 0
  --shadow-far:      0 8px 24px rgba(0, 0, 0, 0.10), 0 24px 64px rgba(0, 0
  --shadow-glow:     none;
}
```

- [ ] **Step 2: Re-run the full token suite**

```
npm run test:run -- src/components/primitives/__tests__/Tokens.test.tsx
```

Expected output: 7 / 7 pass.

Task B5: Refactor tailwind.config.js to read from CSS variables

Files: `src/frontend/tailwind.config.js` (modified)

- [ ] **Step 1: Replace the existing color block with the var() shim**

Open `src/frontend/tailwind.config.js`. **Delete the entire `colors:` object** (the one starting with `primary: { 50: '#eff6ff', …}` and ending after the `border:` block). **Replace it** with:

```
colors: {
  surface: {
    base:    'var(--surface-base)',
    raised:  'var(--surface-raised)',
    overlay: 'var(--surface-overlay)',
    glass:   'var(--surface-glass)',
  },
  ink: {
    primary:   'var(--ink-primary)',
    secondary: 'var(--ink-secondary)',
    tertiary:  'var(--ink-tertiary)',
    muted:     'var(--ink-muted)',
    inverse:   'var(--ink-inverse)',
  },
  accent:      'var(--accent)',
  'accent-soft':'var(--accent-soft)',
  cool:        'var(--cool)',
  warm:        'var(--warm)',
  violet:      'var(--violet)',
  signal: {
    success: 'var(--signal-success)',
    warning: 'var(--signal-warning)',
    danger:  'var(--signal-danger)',
    info:    'var(--signal-info)',
  },
  ring: {
    focus: 'var(--ring-focus)',
  },
  // Reset transparent + currentColor (Tailwind expects these)
  transparent: 'transparent',
  current:     'currentColor',
  white:       '#ffffff',
  black:       '#000000',
},
```

- [ ] **Step 2: Replace the `boxShadow:` block with var() references**

In the same file, replace the `boxShadow:` object with:

```
boxShadow: {
  near: 'var(--shadow-near)',
  far:  'var(--shadow-far)',
  glow: 'var(--shadow-glow)',
  none: 'none',
},
```

- [ ] **Step 3: Replace the `borderRadius:` block**

Replace the existing `borderRadius:` extend with:

```
borderRadius: {
  sm: 'var(--radius-sm)',
  md: 'var(--radius-md)',
  lg: 'var(--radius-lg)',
  xl: 'var(--radius-xl)',
},
```

- [ ] **Step 4: Verify Tailwind builds**

```
npm run build
```

Expected output: Build succeeds. **It is expected** that pages referencing the deleted `primary-500` / `accent-500` classes now produce broken styles — those are repaired in Plan 2. The *build itself* must succeed (Tailwind doesn't error on unknown classes; it just emits no CSS for them).

Task B6: Wire `<body>` data attributes on bootstrap

Files: `src/frontend/index.html` (modified), `src/frontend/src/main.tsx` (modified)

- [ ] **Step 1: Set the operator default on `<body>` in index.html**

Open `src/frontend/index.html`. In the `<body>` opening tag, replace:

```
<body>
```

with:

```
<body data-surface="operator" data-theme="dark">
```

In the same file, **delete** the inline critical CSS that hard-codes `background-color: #0f172a` and `color: #f8fafc` and `font-family: -apple-system, BlinkMacSystemFont, ...`. Replace those rules with:

```
<style>
  html, body {
    margin: 0;
    padding: 0;
    min-height: 100vh;
    background-color: var(--surface-base);
    color: var(--ink-primary);
    font-family: 'Mona Sans Variable', system-ui, sans-serif;
  }
  /* Loading indicator shown until React mounts */
  #app-loading {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 100vh;
    gap: 1rem;
  }
  #app-loading .spinner {
    width: 48px;
    height: 48px;
    border: 4px solid var(--surface-overlay);
    border-top-color: var(--accent);
    border-radius: 50%;
    animation: spin 1s linear infinite;
  }
</style>
```

- [ ] **Step 2: Verify the dev server renders without color regression**

```
npm run dev -- --port 3091 --host 127.0.0.1 &
sleep 3
curl -fsS http://127.0.0.1:3091 | grep -E 'data-surface=|var\(--surface' |
kill %1
```

Expected output: at least one match showing `data-surface="operator"`.

Task B7: Delete the legacy duplicate light-theme block in index.css

Files: `src/frontend/src/index.css` (modified)

- [ ] **Step 1: Remove the legacy CSS-vars block**

Open `src/frontend/src/index.css`. Find the `:root {` block that starts with `color-scheme: light dark;` and contains `--color-bg-primary: #fbf8f1;` (this is the *legacy* theme, now superseded by `tokens.css`). **Delete the entire `:root {…}` block and any subsequent `[data-theme="dark"] {…}`, `.dark {…}`, or `html.dark {…}` blocks** that re-declare these legacy variables. The file should retain only: - The Tailwind directives at top - The `@import './styles/tokens.css';` line - Any remaining utility classes that don't redefine tokens

- [ ] **Step 2: Verify Vitest still passes the token suite**

```
npm run test:run -- src/components/primitives/__tests__/Tokens.test.tsx
```

Expected output: 7 / 7 pass — no regression.

- [ ] **Step 3: Verify build still succeeds**

```
npm run build
```

Expected output: build succeeds.

Task B8: Commit the token foundation

- [ ] **Step 1: Stage and commit**

```
 git add src/frontend/src/styles/tokens.css \
        src/frontend/src/index.css \
        src/frontend/index.html \
        src/frontend/tailwind.config.js \
        src/frontend/src/components/primitives/__tests__/Tokens.test.tsx
git commit -m "$(cat <<'EOF'
feat(redesign): introduce single-source design tokens

Adds src/frontend/src/styles/tokens.css as the canonical token store
with three palette scopes — [data-surface=brand] for Aurora dark,
[data-surface=operator] for Frosted Graphite dark, and
[data-surface=operator][data-theme=light] for the soft-light fallback.

tailwind.config.js becomes a thin shim that reads var(--*); legacy
primary blue (#3b82f6) and fuchsia accent (#d946ef) classes are deleted.
The duplicate :root block in index.css is removed in favour of the
single tokens import.

Token resolution is verified by Vitest (7/7 cases for the three
palettes). No page reflow yet — that's Plan 2.

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Phase C — Font loading (commit 3)

Task C1: Install @fontsource packages

Files: `src/frontend/package.json`, `package-lock.json` (modified)

- [ ] **Step 1: Install the three font packages**

```
cd src/frontend
npm install \
  @fontsource-variable/mona-sans@^5 \
  @fontsource-variable/jetbrains-mono@^5 \
  @fontsource/dm-serif-display@^5
```

Expected output: `added 3 packages` (or `up to date` if mirror prefilled).

- [ ] **Step 2: Verify the packages resolve**

```
ls node_modules/@fontsource-variable/mona-sans/
ls node_modules/@fontsource-variable/jetbrains-mono/
ls node_modules/@fontsource/dm-serif-display/
```

Expected output: each shows files including `index.css` and a `files/` subdirectory with `.woff2` assets.

Task C2: Wire core font imports in main.tsx

Files: `src/frontend/src/main.tsx` (modified)

- [ ] **Step 1: Add the imports at the top of main.tsx**

Open `src/frontend/src/main.tsx`. Above the existing React imports, add:

```
// Self-hosted font faces — loaded once for the entire app.
// DM Serif Display is NOT imported here; the BrandShell lazy-loads it
// only when a brand-surface route mounts (Phase E task E5).
import '@fontsource-variable/mona-sans';
import '@fontsource-variable/jetbrains-mono';
```

- [ ] **Step 2: Verify the dev server still boots**

```
npm run dev -- --port 3091 --host 127.0.0.1 &
sleep 3
curl -fsS -o /dev/null http://127.0.0.1:3091 && echo OK
kill %1
```

Expected output: `OK`.

Task C3: Add @font-face fallback metric overrides

Files: `src/frontend/src/styles/tokens.css` (modified)

- [ ] **Step 1: Append fallback metric overrides**

Append to `src/frontend/src/styles/tokens.css`:

```
/* ------------------------------------------------------------------
   Fallback metric overrides — match system-font metrics to
   Mona Sans + JetBrains Mono so the swap-in doesn't shift layout.
   Numbers are computed against Söhne / Geist proxies; if the
   visual diff in Plan 3 shows shift, re-tune ascent-override and
   size-adjust here.
   ------------------------------------------------------------------ */

@font-face {
  font-family: 'Mona Sans Fallback';
  src: local('Söhne'), local('Inter Variable'), local('Helvetica Neue'), l
  ascent-override: 92%;
  descent-override: 22%;
  line-gap-override: 0%;
  size-adjust: 99%;
}

@font-face {
  font-family: 'JetBrains Mono Fallback';
  src: local('SF Mono'), local('Menlo'), local('Consolas'), local('Liberat
  ascent-override: 88%;
  descent-override: 22%;
  line-gap-override: 0%;
  size-adjust: 100%;
}

/* ------------------------------------------------------------------
   Body / display / mono families.
   ------------------------------------------------------------------ */

:root {
  --font-sans:    'Mona Sans Variable', 'Mona Sans Fallback', system-ui, s
  --font-mono:    'JetBrains Mono Variable', 'JetBrains Mono Fallback', ui
  --font-display: 'DM Serif Display', Georgia, serif;
}

body {
  font-family: var(--font-sans);
  font-weight: 380;
```

```
  font-feature-settings: 'cv11', 'ss01';
  font-optical-sizing: auto;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
code, kbd, pre, samp, .font-mono {
  font-family: var(--font-mono);
  font-feature-settings: 'liga' 0;
}
.tabular-nums {
  font-variant-numeric: tabular-nums slashed-zero;
}
```

- [ ] **Step 2: Verify Vitest token tests still pass**

```
npm run test:run -- src/components/primitives/__tests__/Tokens.test.tsx
```

Expected output: 7 / 7 pass.

Task C4: Add motion.css for global motion + reduced-motion enforcement

Files: - Create: `src/frontend/src/styles/motion.css`

- [ ] **Step 1: Create the motion stylesheet**

Create `src/frontend/src/styles/motion.css`:

```
/* ============================================================
   Global motion baseline + reduced-motion enforcement.
   Per spec section 7.5; verified by Plan 3 a11y sweep.
   ============================================================ */

@keyframes aurora-drift {
  0%   { transform: translate(-2%, -1%) rotate(0deg);  }
  50%  { transform: translate( 2%,  1%) rotate(0.4deg); }
  100% { transform: translate(-2%, -1%) rotate(0deg);  }
}

@keyframes skeleton-shimmer {
  0%   { background-position: -200% 0; }
  100% { background-position:  200% 0; }
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 80ms !important;
    scroll-behavior: auto !important;
  }
  .aurora-glow,
  .skeleton-shimmer,
  .sparkline-draw {
    animation: none !important;
  }
  .motion-transform {
```

```
    transform: none !important;
  }
}
```

- [ ] **Step 2: Import motion.css from index.css**

Open `src/frontend/src/index.css` and add a second import below the tokens import:

```
@import './styles/tokens.css';
@import './styles/motion.css';
```

Task C5: Verify font behavior end-to-end

Files: none — verification only.

- [ ] **Step 1: Build and preview**

```
npm run build
nohup npm run preview -- --port 4173 --host 127.0.0.1 > /tmp/preview-fonts
echo $! > /tmp/preview-fonts.pid
sleep 4
```

- [ ] **Step 2: Verify Mona Sans and JetBrains Mono load (not DM Serif Display)**

```
curl -fsS http://127.0.0.1:4173/ | grep -oE 'mona-sans-[^"]*\.woff2' | hea
curl -fsS http://127.0.0.1:4173/ | grep -oE 'jetbrains-mono-[^"]*\.woff2'
curl -fsS http://127.0.0.1:4173/ | grep -oE 'dm-serif-display' | head -1
```

Expected output: at least one `.woff2` URL for each of the first two, **empty** for `dm-serif-display` (it lazy-loads in Phase E).

- [ ] **Step 3: Stop the preview**

```
kill $(cat /tmp/preview-fonts.pid) 2>/dev/null
rm -f /tmp/preview-fonts.pid
```

Task C6: Commit fonts

- [ ] **Step 1: Stage and commit**

```
 git add src/frontend/package.json src/frontend/package-lock.json \
        src/frontend/src/main.tsx \
        src/frontend/src/styles/tokens.css \
        src/frontend/src/styles/motion.css \
        src/frontend/src/index.css
git commit -m "$(cat <<'EOF'
feat(redesign): self-host Mona Sans + JetBrains Mono, prep DM Serif Displa

Installs @fontsource-variable/{mona-sans,jetbrains-mono} and the static
@fontsource/dm-serif-display package. Mona Sans + JetBrains Mono load
globally via main.tsx imports; DM Serif Display is NOT imported eagerly
and will be lazy-loaded by BrandShell in Phase E.

Adds @font-face fallback metric overrides (Mona Sans Fallback,
JetBrains Mono Fallback) so the swap-in does not shift layout.
Introduces src/frontend/src/styles/motion.css with the global keyframes
and the prefers-reduced-motion enforcement rule.

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Phase D — Primitives (commit 4)

This phase builds 11 primitives (Surface, Button, Card, Input, Badge, Tabs, Dialog, Skeleton, Toast, Table, GradientField). Each follows the same shape: small `[TDD]` cycle for props/states, then commit. Pages are not yet importing these — the legacy `ui/Button` etc. stays in place; we redirect imports in Task D-final.

> **Test pattern.** Every primitive's test file follows the same scaffold: `tsx import { render, screen } from '@testing-library/react'; import { describe, it, expect } from 'vitest'; // import { ComponentName } from '../ComponentName';`

Task D1: [TDD] Surface primitive

Files: - Create: `src/frontend/src/components/primitives/Surface.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Surface.test.tsx`

- [ ] **Step 1: Write the failing test**

```
// src/frontend/src/components/primitives/__tests__/Surface.test.tsx
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import { Surface } from '../Surface';

describe('Surface', () => {
  it('renders children', () => {
    render(<Surface data-testid="s">hi</Surface>);
    expect(screen.getByTestId('s')).toHaveTextContent('hi');
  });
  it('applies elevation="raised" → bg-surface-raised class', () => {
    render(<Surface elevation="raised" data-testid="s" />);
    expect(screen.getByTestId('s').className).toMatch(/bg-surface-raised/)
  });
  it('applies elevation="overlay" → bg-surface-overlay class', () => {
    render(<Surface elevation="overlay" data-testid="s" />);
    expect(screen.getByTestId('s').className).toMatch(/bg-surface-overlay/
  });
  it('default elevation is base', () => {
    render(<Surface data-testid="s" />);
    expect(screen.getByTestId('s').className).toMatch(/bg-surface-base/);
  });
});
```

- [ ] **Step 2: Run, confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Surface.test.tsx
```

Expected: 4 fails (`Surface` not exported).

- [ ] **Step 3: Implement Surface**

Create `src/frontend/src/components/primitives/Surface.tsx`:

```
import { forwardRef, HTMLAttributes } from 'react';
import { clsx } from 'clsx';

export type SurfaceElevation = 'base' | 'raised' | 'overlay';

export interface SurfaceProps extends HTMLAttributes<HTMLDivElement> {
  elevation?: SurfaceElevation;
}

const elevationClass: Record<SurfaceElevation, string> = {
  base:    'bg-surface-base',
  raised:  'bg-surface-raised',
  overlay: 'bg-surface-overlay',
};

export const Surface = forwardRef<HTMLDivElement, SurfaceProps>(
  ({ elevation = 'base', className, ...rest }, ref) => (
    <div ref={ref} className={clsx(elevationClass[elevation], 'text-ink-pr
  ),
);

Surface.displayName = 'Surface';
```

- [ ] **Step 4: Run, confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Surface.test.tsx
```

Expected: 4 / 4 pass.

Task D2: [TDD] Button primitive

Files: - Create: `src/frontend/src/components/primitives/Button.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Button.test.tsx`

- [ ] **Step 1: Write the failing test**

```
// src/frontend/src/components/primitives/__tests__/Button.test.tsx
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library/react';
import { Button } from '../Button';

describe('Button', () => {
  it('renders children and fires onClick', () => {
    const onClick = vi.fn();
    render(<Button onClick={onClick}>Save</Button>);
    fireEvent.click(screen.getByRole('button', { name: /save/i }));
    expect(onClick).toHaveBeenCalledOnce();
  });
  it('variant="primary" uses accent background', () => {
    render(<Button variant="primary">go</Button>);
    expect(screen.getByRole('button').className).toMatch(/bg-accent/);
  });
  it('variant="ghost" is text-only', () => {
    render(<Button variant="ghost">go</Button>);
    expect(screen.getByRole('button').className).not.toMatch(/bg-accent/);
  });
  it('variant="danger" uses signal-danger background', () => {
    render(<Button variant="danger">remove</Button>);
    expect(screen.getByRole('button').className).toMatch(/bg-signal-danger
  });
  it('disabled prop adds aria-disabled and disables onClick', () => {
    const onClick = vi.fn();
    render(<Button disabled onClick={onClick}>x</Button>);
    fireEvent.click(screen.getByRole('button'));
    expect(onClick).not.toHaveBeenCalled();
    expect(screen.getByRole('button')).toHaveAttribute('aria-disabled', 't
  });
  it('loading prop replaces children with a Skeleton + retains accessible
    render(<Button loading aria-label="Save">Save</Button>);
    expect(screen.getByRole('button', { name: /save/i })).toBeInTheDocumen
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Button.test.tsx
```

Expected: 6 fails.

- [ ] **Step 3: Implement Button**

Create `src/frontend/src/components/primitives/Button.tsx`:

```
import { forwardRef, ButtonHTMLAttributes, ReactNode } from 'react';
import { clsx } from 'clsx';

export type ButtonVariant = 'primary' | 'secondary' | 'ghost' | 'danger';
export type ButtonSize = 'sm' | 'md' | 'lg';

export interface ButtonProps extends Omit<ButtonHTMLAttributes<HTMLButtonE
  variant?: ButtonVariant;
  size?: ButtonSize;
  loading?: boolean;
  disabled?: boolean;
  leadingIcon?: ReactNode;
  trailingIcon?: ReactNode;
}

const base =
  'inline-flex items-center justify-center gap-2 rounded-md font-[520] ' +
  'transition-[transform,background-color,box-shadow] duration-[80ms] ease
  'focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
  'active:scale-[0.98] motion-transform select-none';

const sizeClass: Record<ButtonSize, string> = {
  sm: 'h-8  px-3 text-xs',
  md: 'h-10 px-4 text-sm',
  lg: 'h-12 px-5 text-base',
};

const variantClass: Record<ButtonVariant, string> = {
  primary:
    'bg-accent text-ink-inverse hover:brightness-110 ' +
    'shadow-near hover:shadow-far',
  secondary:
    'bg-surface-raised text-ink-primary border border-surface-overlay hove
  ghost:
    'bg-transparent text-ink-primary hover:bg-surface-raised',
  danger:
    'bg-signal-danger text-ink-inverse hover:brightness-110 shadow-near',
};
```

```
export const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  ({ variant = 'primary', size = 'md', loading, disabled, leadingIcon, tra
    const isInert = disabled || loading;
    return (
      <button
        ref={ref}
        type={rest.type ?? 'button'}
        aria-disabled={isInert || undefined}
        aria-busy={loading || undefined}
        disabled={isInert}
        className={clsx(base, sizeClass[size], variantClass[variant], isIn
        {...rest}
      >
        {leadingIcon && <span aria-hidden>{leadingIcon}</span>}
        <span>{children}</span>
        {trailingIcon && <span aria-hidden>{trailingIcon}</span>}
      </button>
    );
  },
);

Button.displayName = 'Button';
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Button.test.tsx
```

Expected: 6 / 6 pass.

Task D3: [TDD] Card primitive

Files: - Create: `src/frontend/src/components/primitives/Card.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Card.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/primitives/__tests__/Card.test.tsx
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import { Card } from '../Card';

describe('Card', () => {
  it('renders children', () => {
    render(<Card data-testid="c">x</Card>);
    expect(screen.getByTestId('c')).toHaveTextContent('x');
  });
  it('hoverable adds the hover class', () => {
    render(<Card hoverable data-testid="c">x</Card>);
    expect(screen.getByTestId('c').className).toMatch(/hover:translate-y-/
  });
  it('density="dense" → tighter padding', () => {
    render(<Card density="dense" data-testid="c">x</Card>);
    expect(screen.getByTestId('c').className).toMatch(/p-3/);
  });
  it('default density is comfortable (p-5)', () => {
    render(<Card data-testid="c">x</Card>);
    expect(screen.getByTestId('c').className).toMatch(/p-5/);
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Card.test.tsx
```

Expected: 4 fails.

- [ ] **Step 3: Implement**

Create `src/frontend/src/components/primitives/Card.tsx`:

```
import { forwardRef, HTMLAttributes } from 'react';
import { clsx } from 'clsx';

export type CardDensity = 'dense' | 'comfortable' | 'roomy';

export interface CardProps extends HTMLAttributes<HTMLDivElement> {
  hoverable?: boolean;
  density?: CardDensity;
}

const densityClass: Record<CardDensity, string> = {
  dense:        'p-3',
  comfortable:  'p-5',
  roomy:        'p-7',
};

const base =
  'rounded-lg bg-surface-raised text-ink-primary border border-surface-ove
  'shadow-near transition-[transform,box-shadow] duration-200 ease-out';

const hoverClass =
  'hover:-translate-y-px hover:shadow-far motion-transform';

export const Card = forwardRef<HTMLDivElement, CardProps>(
  ({ hoverable, density = 'comfortable', className, ...rest }, ref) => (
    <div ref={ref} className={clsx(base, densityClass[density], hoverable
  ),
);

Card.displayName = 'Card';
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Card.test.tsx
```

Expected: 4 / 4 pass.

Task D4: [TDD] Input primitive

Files: - Create: `src/frontend/src/components/primitives/Input.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Input.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/primitives/__tests__/Input.test.tsx
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library/react';
import { Input } from '../Input';

describe('Input', () => {
  it('renders with label and forwards ref', () => {
    render(<Input label="Email" name="email" />);
    expect(screen.getByLabelText('Email')).toBeInTheDocument();
  });
  it('forwards onChange', () => {
    const onChange = vi.fn();
    render(<Input label="x" onChange={onChange} />);
    fireEvent.change(screen.getByLabelText('x'), { target: { value: 'hi' }
    expect(onChange).toHaveBeenCalled();
  });
  it('renders error helper text and aria-invalid', () => {
    render(<Input label="x" error="required" />);
    expect(screen.getByText('required')).toBeInTheDocument();
    expect(screen.getByLabelText('x')).toHaveAttribute('aria-invalid', 'tr
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Input.test.tsx
```

Expected: 3 fails.

- [ ] **Step 3: Implement**

Create `src/frontend/src/components/primitives/Input.tsx`:

```
import { forwardRef, InputHTMLAttributes, useId } from 'react';
import { clsx } from 'clsx';

export interface InputProps extends InputHTMLAttributes<HTMLInputElement>
  label: string;
  hint?: string;
  error?: string;
}

export const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, hint, error, className, id, ...rest }, ref) => {
    const reactId = useId();
    const inputId = id ?? reactId;
    return (
      <label htmlFor={inputId} className="flex flex-col gap-1 text-sm">
        <span className="text-ink-secondary">{label}</span>
        <input
          id={inputId}
          ref={ref}
          aria-invalid={!!error || undefined}
          aria-describedby={(error || hint) ? `${inputId}-help` : undefine
          className={clsx(
            'h-10 px-3 rounded-md bg-surface-base text-ink-primary placeho
            'border border-surface-overlay focus-visible:outline-none focu
            error && 'border-signal-danger',
            className,
          )}
          {...rest}
        />
        {(error || hint) && (
          <span id={`${inputId}-help`} className={clsx('text-xs', error ?
            {error ?? hint}
          </span>
        )}
      </label>
    );
  },
```

```
);

Input.displayName = 'Input';
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Input.test.tsx
```

Expected: 3 / 3 pass.

Task D5: [TDD] Badge primitive

Files: - Create: `src/frontend/src/components/primitives/Badge.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Badge.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/primitives/__tests__/Badge.test.tsx
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import { Badge } from '../Badge';

describe('Badge', () => {
  it('default tone uses ink-secondary', () => {
    render(<Badge>label</Badge>);
    expect(screen.getByText('label').className).toMatch(/text-ink-secondar
  });
  it.each(['critical', 'high', 'medium', 'low'] as const)('severity %s ren
    render(<Badge severity={sev}>{sev}</Badge>);
    expect(screen.getByText(sev)).toBeInTheDocument();
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Badge.test.tsx
```

Expected: 5 fails.

- [ ] **Step 3: Implement**

Create `src/frontend/src/components/primitives/Badge.tsx`:

```
import { HTMLAttributes } from 'react';
import { clsx } from 'clsx';

export type BadgeSeverity = 'critical' | 'high' | 'medium' | 'low';

export interface BadgeProps extends HTMLAttributes<HTMLSpanElement> {
  severity?: BadgeSeverity;
}

const severityClass: Record<BadgeSeverity, string> = {
  critical: 'bg-signal-danger/15  text-signal-danger  border-signal-danger
  high:     'bg-warm/15           text-warm           border-warm/30',
  medium:   'bg-signal-warning/15 text-signal-warning border-signal-warnin
  low:      'bg-ink-muted/20      text-ink-tertiary   border-ink-muted/30'
};

const base =
  'inline-flex items-center gap-1 rounded-sm border px-2 py-0.5 ' +
  'text-xs font-[520] uppercase tracking-wide';

export const Badge = ({ severity, className, ...rest }: BadgeProps) => (
  <span
    className={clsx(
      base,
      severity ? severityClass[severity] : 'bg-surface-raised text-ink-sec
      className,
    )}
    {...rest}
  />
);
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Badge.test.tsx
```

Expected: 5 / 5 pass.

Task D6: [TDD] Tabs primitive (with sliding indicator)

Files: - Create: `src/frontend/src/components/primitives/Tabs.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Tabs.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/primitives/__tests__/Tabs.test.tsx
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library/react';
import { Tabs } from '../Tabs';

describe('Tabs', () => {
  it('renders the supplied items and marks the active one', () => {
    render(
      <Tabs
        items={[{ id: 'a', label: 'Alpha' }, { id: 'b', label: 'Beta' }]}
        active="b"
        onChange={() => {}}
      />,
    );
    expect(screen.getByRole('tab', { name: 'Beta' })).toHaveAttribute('ari
  });
  it('fires onChange with the clicked id', () => {
    const onChange = vi.fn();
    render(
      <Tabs
        items={[{ id: 'a', label: 'Alpha' }, { id: 'b', label: 'Beta' }]}
        active="a"
        onChange={onChange}
      />,
    );
    fireEvent.click(screen.getByRole('tab', { name: 'Beta' }));
    expect(onChange).toHaveBeenCalledWith('b');
  });
  it('arrow keys navigate', () => {
    const onChange = vi.fn();
    render(
      <Tabs
        items={[{ id: 'a', label: 'A' }, { id: 'b', label: 'B' }, { id: 'c
        active="a"
        onChange={onChange}
      />,
    );
    fireEvent.keyDown(screen.getByRole('tab', { name: 'A' }), { key: 'Arro
```

```
    expect(onChange).toHaveBeenCalledWith('b');
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Tabs.test.tsx
```

Expected: 3 fails.

- [ ] **Step 3: Implement**

Create `src/frontend/src/components/primitives/Tabs.tsx`:

```
import { KeyboardEvent } from 'react';
import { clsx } from 'clsx';
import { motion } from 'framer-motion';

export interface TabItem {
  id: string;
  label: string;
  disabled?: boolean;
}

export interface TabsProps {
  items: TabItem[];
  active: string;
  onChange: (id: string) => void;
  className?: string;
  size?: 'sm' | 'md';
}

export const Tabs = ({ items, active, onChange, className, size = 'md' }:
  const onKey = (e: KeyboardEvent<HTMLButtonElement>) => {
    const idx = items.findIndex((t) => t.id === active);
    if (idx < 0) return;
    let next = idx;
    if (e.key === 'ArrowRight') next = Math.min(items.length - 1, idx + 1)
    if (e.key === 'ArrowLeft')  next = Math.max(0, idx - 1);
    if (e.key === 'Home')       next = 0;
    if (e.key === 'End')        next = items.length - 1;
    if (next !== idx) {
      e.preventDefault();
      onChange(items[next].id);
    }
  };

  return (
    <div role="tablist" className={clsx('flex items-stretch border-b borde
      {items.map((t) => {
        const selected = t.id === active;
        return (
          <button
```

```
            key={t.id}
            role="tab"
            aria-selected={selected}
            aria-disabled={t.disabled || undefined}
            disabled={t.disabled}
            tabIndex={selected ? 0 : -1}
            onClick={() => !t.disabled && onChange(t.id)}
            onKeyDown={onKey}
            className={clsx(
              'relative px-4 py-2 text-sm font-[520] transition-colors',
              size === 'sm' ? 'h-8' : 'h-10',
              selected ? 'text-ink-primary' : 'text-ink-tertiary hover:tex
              t.disabled && 'opacity-50 pointer-events-none',
              'focus-visible:outline-none focus-visible:ring-2 focus-visib
            )}
          >
            {t.label}
            {selected && (
              <motion.span
                layoutId="tabs-indicator"
                transition={{ type: 'spring', stiffness: 420, damping: 32
                className="absolute left-2 right-2 -bottom-px h-[2px] bg-a
              />
            )}
          </button>
        );
      })}
    </div>
  );
};
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Tabs.test.tsx
```

Expected: 3 / 3 pass.

Task D7: [TDD] Dialog primitive

Files: - Create: `src/frontend/src/components/primitives/Dialog.tsx` - Create: `src/frontend/src/components/primitives/__tests__/Dialog.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/primitives/__tests__/Dialog.test.tsx
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library/react';
import { Dialog } from '../Dialog';

describe('Dialog', () => {
  it('does not render when open=false', () => {
    render(<Dialog open={false} title="t" onClose={() => {}}>x</Dialog>);
    expect(screen.queryByRole('dialog')).toBeNull();
  });
  it('renders title and content when open', () => {
    render(<Dialog open title="Confirm" onClose={() => {}}>Body</Dialog>);
    expect(screen.getByRole('dialog')).toHaveTextContent('Confirm');
    expect(screen.getByRole('dialog')).toHaveTextContent('Body');
  });
  it('Escape calls onClose', () => {
    const onClose = vi.fn();
    render(<Dialog open title="t" onClose={onClose}>x</Dialog>);
    fireEvent.keyDown(screen.getByRole('dialog'), { key: 'Escape' });
    expect(onClose).toHaveBeenCalled();
  });
  it('clicking the scrim calls onClose', () => {
    const onClose = vi.fn();
    render(<Dialog open title="t" onClose={onClose}>x</Dialog>);
    fireEvent.click(screen.getByTestId('dialog-scrim'));
    expect(onClose).toHaveBeenCalled();
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/primitives/__tests__/Dialog.test.tsx
```

Expected: 4 fails.

- [ ] **Step 3: Implement**

Create `src/frontend/src/components/primitives/Dialog.tsx`:

```
import { ReactNode, useEffect, useRef } from 'react';
import { createPortal } from 'react-dom';
import { motion, AnimatePresence } from 'framer-motion';
import { clsx } from 'clsx';

export interface DialogProps {
  open: boolean;
  title: string;
  onClose: () => void;
  children: ReactNode;
  width?: 'sm' | 'md' | 'lg';
}

const widthClass: Record<NonNullable<DialogProps['width']>, string> = {
  sm: 'max-w-md',
  md: 'max-w-lg',
  lg: 'max-w-3xl',
};

export const Dialog = ({ open, title, onClose, children, width = 'md' }: D
  const ref = useRef<HTMLDivElement>(null);
  useEffect(() => {
    if (!open) return;
    const onKey = (e: KeyboardEvent) => { if (e.key === 'Escape') onClose(
    window.addEventListener('keydown', onKey);
    return () => window.removeEventListener('keydown', onKey);
  }, [open, onClose]);
  if (typeof document === 'undefined') return null;
  return createPortal(
    <AnimatePresence>
      {open && (
        <div
          role="dialog"
          aria-modal="true"
          aria-label={title}
          ref={ref}
          tabIndex={-1}
          onKeyDown={(e) => { if (e.key === 'Escape') onClose(); }}
          className="fixed inset-0 z-50 flex items-center justify-center p
```

```
        >
          <motion.div
            data-testid="dialog-scrim"
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            transition={{ duration: 0.18, ease: [0.16, 1, 0.3, 1] }}
            className="absolute inset-0 bg-surface-base/70 backdrop-blur-s
            onClick={onClose}
          />
          <motion.div
            initial={{ opacity: 0, y: 8 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0, y: 0 }}
            transition={{ duration: 0.18, ease: [0.16, 1, 0.3, 1] }}
            className={clsx('relative bg-surface-overlay text-ink-primary
          >
            <h2 className="text-lg font-[680] mb-4">{title}</h2>
            <div>{children}</div>
          </motion.div>
        </div>
      )}
    </AnimatePresence>,
    document.body,
  );
};
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/primitives/__tests__/Dialog.test.tsx
```

Expected: 4 / 4 pass.

Task D8: [Visual] Skeleton primitive

Files: - Create: `src/frontend/src/components/primitives/Skeleton.tsx`

- [ ] **Step 1: Implement (no TDD — pure visual)**

Create `src/frontend/src/components/primitives/Skeleton.tsx`:

```
import { HTMLAttributes } from 'react';
import { clsx } from 'clsx';

export interface SkeletonProps extends HTMLAttributes<HTMLDivElement> {
  width?: string | number;
  height?: string | number;
  rounded?: 'sm' | 'md' | 'lg' | 'full';
}

const roundedClass = {
  sm:   'rounded-sm',
  md:   'rounded-md',
  lg:   'rounded-lg',
  full: 'rounded-full',
};

export const Skeleton = ({ width = '100%', height = 14, rounded = 'sm', cl
  <div
    aria-hidden
    className={clsx(
      'skeleton-shimmer bg-surface-overlay',
      roundedClass[rounded],
      className,
    )}
    style={{
      width,
      height,
      backgroundImage:
        'linear-gradient(90deg, transparent 0%, rgba(255,255,255,0.06) 50%
      backgroundSize: '200% 100%',
      animation: 'skeleton-shimmer 1400ms linear infinite',
      ...style,
    }}
    {...rest}
  />
);
```

- [ ] **Step 2: Smoke test that the module loads**

```
npm run test:run -- --run src/components/primitives/Skeleton.tsx
```

Expected: tests don't reference it yet, but `tsc` should be happy. We verify in build.

```
npm run build
```

Expected output: build succeeds.

Task D9: [Visual] Toast primitive

Files: - Create: `src/frontend/src/components/primitives/Toast.tsx`

- [ ] **Step 1: Implement**

Create `src/frontend/src/components/primitives/Toast.tsx`:

```
import { ReactNode } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { clsx } from 'clsx';

export type ToastTone = 'default' | 'success' | 'warning' | 'danger';

export interface ToastProps {
  open: boolean;
  tone?: ToastTone;
  title: string;
  description?: ReactNode;
  onDismiss?: () => void;
}

const toneClass: Record<ToastTone, string> = {
  default: 'border-surface-overlay',
  success: 'border-signal-success/40',
  warning: 'border-signal-warning/40',
  danger:  'border-signal-danger/40',
};

export const Toast = ({ open, tone = 'default', title, description, onDism
  <AnimatePresence>
    {open && (
      <motion.div
        role="status"
        initial={{ opacity: 0, x: 24 }}
        animate={{ opacity: 1, x: 0 }}
        exit={{ opacity: 0, x: 16 }}
        transition={{ type: 'spring', stiffness: 380, damping: 28 }}
        className={clsx(
          'fixed bottom-6 right-6 z-50 max-w-sm rounded-lg border p-4 bg-s
          toneClass[tone],
        )}
      >
        <div className="flex items-start gap-3">
          <div className="flex-1">
            <div className="text-sm font-[520]">{title}</div>
            {description && <div className="text-xs text-ink-tertiary mt-1
```

```
          </div>
          {onDismiss && (
            <button
              type="button"
              aria-label="Dismiss"
              onClick={onDismiss}
              className="text-ink-tertiary hover:text-ink-primary"
            >
              ×
            </button>
          )}
        </div>
      </motion.div>
    )}
  </AnimatePresence>
);
```

- [ ] **Step 2: Build to confirm types**

```
npm run build
```

Expected: build succeeds.

Task D10: [Visual] Table primitive

Files: - Create: `src/frontend/src/components/primitives/Table.tsx`

- [ ] **Step 1: Implement**

Create `src/frontend/src/components/primitives/Table.tsx`:

```
import { HTMLAttributes, ReactNode, ThHTMLAttributes, TdHTMLAttributes } f
import { clsx } from 'clsx';

export const Table = ({ className, ...rest }: HTMLAttributes<HTMLTableElem
  <table className={clsx('w-full text-sm tabular-nums', className)} {...re
);
export const THead = ({ className, ...rest }: HTMLAttributes<HTMLTableSect
  <thead className={clsx('text-ink-tertiary uppercase tracking-wide text-x
);
export const TBody = (props: HTMLAttributes<HTMLTableSectionElement>) => <
export const Tr = ({ className, ...rest }: HTMLAttributes<HTMLTableRowElem
  <tr className={clsx('border-b border-surface-overlay/40 hover:bg-surface
);
export const Th = ({ className, ...rest }: ThHTMLAttributes<HTMLTableHeade
  <th className={clsx('text-left font-[520] py-2 px-3', className)} {...re
);
export const Td = ({ className, ...rest }: TdHTMLAttributes<HTMLTableDataC
  <td className={clsx('py-2 px-3 text-ink-primary', className)} {...rest}
);

export interface DataTableColumn<T> {
  key: keyof T | string;
  header: ReactNode;
  cell: (row: T) => ReactNode;
  align?: 'left' | 'right' | 'center';
}

export function DataTable<T>({ rows, columns, empty }: { rows: T[]; column
  return (
    <Table>
      <THead>
        <Tr>
          {columns.map((c) => (
            <Th key={String(c.key)} className={clsx(c.align === 'right' &&
              {c.header}
            </Th>
          ))}
        </Tr>
      </THead>
```

```
      <TBody>
        {rows.length === 0 && empty ? (
          <Tr>
            <Td colSpan={columns.length} className="text-center text-ink-t
          </Tr>
        ) : (
          rows.map((row, i) => (
            <Tr key={i}>
              {columns.map((c) => (
                <Td key={String(c.key)} className={clsx(c.align === 'right
                  {c.cell(row)}
                </Td>
              ))}
            </Tr>
          ))
        )}
      </TBody>
    </Table>
  );
}
```

- [ ] **Step 2: Build to confirm types**

```
npm run build
```

Expected: build succeeds.

Task D11: [Visual] GradientField (Aurora-glow background utility)

Files: - Create: `src/frontend/src/components/primitives/GradientField.tsx`

- [ ] **Step 1: Implement**

Create `src/frontend/src/components/primitives/GradientField.tsx`:

```
import { HTMLAttributes } from 'react';
import { clsx } from 'clsx';

export interface GradientFieldProps extends HTMLAttributes<HTMLDivElement>
  intensity?: 'subtle' | 'full';
}

/**
 * Aurora-glow gradient. Used by BrandShell as a fixed background.
 * Full intensity = brand routes; subtle = optional decorative use elsewhe
 * Animation freezes under prefers-reduced-motion.
 */
export const GradientField = ({ intensity = 'full', className, style, ...r
  <div
    aria-hidden
    className={clsx('aurora-glow pointer-events-none fixed inset-[-20%] -z
    style={{
      backgroundImage: [
        'radial-gradient(60vmax 60vmax at 30% 20%, var(--warm)   0%, trans
        'radial-gradient(50vmax 50vmax at 70% 80%, var(--cool)   0%, trans
        'radial-gradient(40vmax 40vmax at 50% 50%, var(--violet) 0%, trans
      ].join(', '),
      filter: 'blur(60px) saturate(1.2)',
      opacity: intensity === 'full' ? 0.18 : 0.04,
      animation: intensity === 'full' ? 'aurora-drift 60s linear infinite'
      ...style,
    }}
    {...rest}
  />
);
```

- [ ] **Step 2: Build**

```
npm run build
```

Expected: build succeeds.

Task D12: Primitive barrel export + legacy redirects

Files: - Create: `src/frontend/src/components/primitives/index.ts` - Modify: `src/frontend/src/components/ui/index.ts` (modify to re-export from primitives) - Delete: `src/frontend/src/components/ui/Button.tsx`, `Card.tsx`, `Spinner.tsx`

- [ ] **Step 1: Create the primitives barrel**

Create `src/frontend/src/components/primitives/index.ts`:

```
export { Surface } from './Surface';
export { Button } from './Button';
export { Card } from './Card';
export { Input } from './Input';
export { Badge } from './Badge';
export { Tabs } from './Tabs';
export { Dialog } from './Dialog';
export { Skeleton } from './Skeleton';
export { Toast } from './Toast';
export { Table, THead, TBody, Tr, Th, Td, DataTable } from './Table';
export { GradientField } from './GradientField';

export type { SurfaceProps, SurfaceElevation } from './Surface';
export type { ButtonProps, ButtonVariant, ButtonSize } from './Button';
export type { CardProps, CardDensity } from './Card';
export type { InputProps } from './Input';
export type { BadgeProps, BadgeSeverity } from './Badge';
export type { TabsProps, TabItem } from './Tabs';
export type { DialogProps } from './Dialog';
export type { SkeletonProps } from './Skeleton';
export type { ToastProps, ToastTone } from './Toast';
export type { DataTableColumn } from './Table';
export type { GradientFieldProps } from './GradientField';
```

- [ ] **Step 2: Replace the legacy ui barrel with redirects**

Open `src/frontend/src/components/ui/index.ts` and replace its entire contents with:

```
// Compatibility shim — legacy callers import from components/ui.
// Plan 2 will migrate these imports to components/primitives directly.
export { Button } from '../primitives/Button';
export { Card }   from '../primitives/Card';
// Spinner is gone in the new system; consumers move to <Skeleton /> in Pl
```

- [ ] **Step 3: Delete the legacy primitive files**

```
git rm src/components/ui/Button.tsx src/components/ui/Card.tsx src/compone
```

(Adjust path prefix for the worktree root if needed; the actual paths are `src/frontend/src/components/ui/Button.tsx` etc.)

```
git rm src/frontend/src/components/ui/Button.tsx \
      src/frontend/src/components/ui/Card.tsx \
      src/frontend/src/components/ui/Spinner.tsx
```

- [ ] **Step 4: Run typecheck to find broken Spinner imports**

```
cd src/frontend
npm run typecheck
```

Expected output: a list of files importing `Spinner` from `../ui` or similar. **Note them down for Plan 2** (those are page-level fixes), but for now, replace each `<Spinner />` usage with `<Skeleton width={24} height={24} rounded="full" />`. This may require touching a small number of legacy components — **only fix what's blocking typecheck**, not what's blocking visual correctness.

If typecheck shows N errors, do the smallest possible substitution per file: - `import { Spinner } from '...';` → `import { Skeleton } from '../primitives';` - `<Spinner />` → `<Skeleton width={24} height={24} rounded="full" />`

- [ ] **Step 5: Confirm typecheck and build pass**

```
npm run typecheck
npm run build
```

Expected output: both succeed.

- [ ] **Step 6: Run all primitive tests**

```
npm run test:run -- src/components/primitives/__tests__
```

Expected output: all primitive tests green (Tokens 7, Surface 4, Button 6, Card 4, Input 3, Badge 5, Tabs 3, Dialog 4 = 36 / 36).

Task D13: Commit primitives

- [ ] **Step 1: Stage and commit**

```
git add src/frontend/src/components/primitives \
        src/frontend/src/components/ui \
        src/frontend/src/components
git commit -m "$(cat <<'EOF'
feat(redesign): rebuild UI primitives against design tokens

Adds 11 primitives under src/components/primitives — Surface, Button,
Card, Input, Badge, Tabs, Dialog, Skeleton, Toast, Table, GradientField
— all reading from the token system and following the spec's
density/severity/tone conventions. Each primitive ships TDD coverage
where the contract is testable; Skeleton/Toast/Table/GradientField are
visual-only and verified by build.

Replaces the legacy components/ui/{Button,Card,Spinner}.tsx primitives.
ui/index.ts is now a compatibility shim re-exporting from primitives so
Plan 2 can migrate page-level imports lazily. Spinner is deleted in
favor of Skeleton.

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Phase E — Shells (commit 5)

The shells are the spine of the redesign. BrandShell wraps brand routes (Login, Register, etc.); OperatorShell wraps everything else and contains Sidebar + Topbar + RightRail (where applicable).

Task E1: [TDD] BrandShell — surface attribute + glow

Files: - Create: `src/frontend/src/components/layout/BrandShell.tsx` - Create: `src/frontend/src/components/layout/__tests__/BrandShell.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/layout/__tests__/BrandShell.test.tsx
import { describe, it, expect, beforeEach, afterEach } from 'vitest';
import { render } from '@testing-library/react';
import { BrandShell } from '../BrandShell';

describe('BrandShell', () => {
  beforeEach(() => {
    document.body.removeAttribute('data-surface');
    document.body.removeAttribute('data-theme');
  });
  afterEach(() => {
    document.body.removeAttribute('data-surface');
  });
  it('sets data-surface=brand on body while mounted', () => {
    const { unmount } = render(<BrandShell><div>x</div></BrandShell>);
    expect(document.body.getAttribute('data-surface')).toBe('brand');
    unmount();
    expect(document.body.getAttribute('data-surface')).not.toBe('brand');
  });
  it('renders the GradientField (aurora-glow class)', () => {
    const { container } = render(<BrandShell><div>x</div></BrandShell>);
    expect(container.querySelector('.aurora-glow')).toBeTruthy();
  });
  it('renders children', () => {
    const { getByText } = render(<BrandShell><div>hello</div></BrandShell>
    expect(getByText('hello')).toBeInTheDocument();
  });
});
```

- [ ] **Step 2: Confirm failure**

```
cd src/frontend
npm run test:run -- src/components/layout/__tests__/BrandShell.test.tsx
```

Expected: 3 fails.

- [ ] **Step 3: Implement BrandShell**

Create `src/frontend/src/components/layout/BrandShell.tsx`:

```
import { ReactNode, useEffect } from 'react';
import { GradientField } from '../primitives/GradientField';

let dmSerifLoaded = false;

async function loadDMSerifOnce(): Promise<void> {
  if (dmSerifLoaded) return;
  dmSerifLoaded = true;
  // Lazy import — only mounted on brand routes.
  await import('@fontsource/dm-serif-display/400.css');
}

export const BrandShell = ({ children }: { children: ReactNode }) => {
  useEffect(() => {
    document.body.setAttribute('data-surface', 'brand');
    void loadDMSerifOnce();
    return () => {
      document.body.removeAttribute('data-surface');
    };
  }, []);

  return (
    <div className="relative min-h-screen flex flex-col items-center justi
      <GradientField intensity="full" />
      <main className="relative z-10 w-full max-w-[520px]">{children}</mai
    </div>
  );
};
```

- [ ] **Step 4: Confirm pass**

```
npm run test:run -- src/components/layout/__tests__/BrandShell.test.tsx
```

Expected: 3 / 3 pass.

Task E2: [TDD] OperatorShell — surface + theme + sidebar slot

Files: - Create: `src/frontend/src/components/layout/OperatorShell.tsx` - Create: `src/frontend/src/components/layout/__tests__/OperatorShell.test.tsx`

- [ ] **Step 1: Failing test**

```
// src/frontend/src/components/layout/__tests__/OperatorShell.test.tsx
import { describe, it, expect, beforeEach, afterEach } from 'vitest';
import { render } from '@testing-library/react';
import { OperatorShell } from '../OperatorShell';

describe('OperatorShell', () => {
  beforeEach(() => {
    document.body.removeAttribute('data-surface');
    document.body.removeAttribute('data-theme');
  });
  afterEach(() => {
    document.body.removeAttribute('data-surface');
    document.body.removeAttribute('data-theme');
  });
  it('sets data-surface=operator and data-theme=dark by default', () => {
    render(<OperatorShell><div>x</div></OperatorShell>);
    expect(document.body.getAttribute('data-surface')).toBe('operator');
    expect(document.body.getAttribute('data-theme')).toBe('dark');
  });
  it('honors theme="light"', () => {
    render(<OperatorShell theme="light"><div>x</div></OperatorShell>);
    expect(document.body.getAttribute('data-theme')).toBe('light');
  });
  it('renders the children', () => {
    const { getByText } = render(<OperatorShell><div>hi</div></OperatorShe
    expect(getByText('hi')).toBeInTheDocument();
  });
});
```

- [ ] **Step 2: Confirm failure**

```
npm run test:run -- src/components/layout/__tests__/OperatorShell.test.tsx
```

Expected: 3 fails.

- [ ] **Step 3: Implement OperatorShell**

Create `src/frontend/src/components/layout/OperatorShell.tsx`:

```
import { ReactNode, useEffect } from 'react';
import { Sidebar } from './Sidebar';
import { Topbar } from './Topbar';
import { GradientField } from '../primitives/GradientField';

export interface OperatorShellProps {
  children: ReactNode;
  theme?: 'dark' | 'light';
  /** Optional sticky right-rail content (used by Pattern 4). */
  rightRail?: ReactNode;
  /** Hide the sidebar entirely (e.g., focused detail mode). */
  hideSidebar?: boolean;
}

export const OperatorShell = ({ children, theme = 'dark', rightRail, hideS
  useEffect(() => {
    document.body.setAttribute('data-surface', 'operator');
    document.body.setAttribute('data-theme', theme);
    return () => {
      document.body.removeAttribute('data-surface');
      document.body.removeAttribute('data-theme');
    };
  }, [theme]);

  return (
    <div className="relative min-h-screen flex bg-surface-base text-ink-pr
      <GradientField intensity="subtle" />
      {!hideSidebar && <Sidebar />}
      <div className="flex-1 flex flex-col min-w-0">
        <Topbar />
        <div className="flex-1 flex">
          <main className="flex-1 px-6 py-6 min-w-0">{children}</main>
          {rightRail && (
            <aside className="hidden lg:block w-[280px] shrink-0 border-l
              {rightRail}
            </aside>
          )}
        </div>
      </div>
```

```
    </div>
  );
};
```

- [ ] **Step 4: Implement Sidebar**

Create `src/frontend/src/components/layout/Sidebar.tsx`:

```
import { useState } from 'react';
import { NavLink } from 'react-router-dom';
import { clsx } from 'clsx';

interface NavItem {
  to: string;
  label: string;
  icon?: string;
}

const NAV: NavItem[] = [
  { to: '/dashboard',     label: 'Dashboard' },
  { to: '/scripts',       label: 'Scripts' },
  { to: '/search',        label: 'Search' },
  { to: '/categories',    label: 'Categories' },
  { to: '/documentation', label: 'Documentation' },
  { to: '/chat',          label: 'Chat' },
  { to: '/agentic',       label: 'Agentic' },
  { to: '/analytics',     label: 'Analytics' },
  { to: '/settings',      label: 'Settings' },
];

export const Sidebar = () => {
  const [collapsed, setCollapsed] = useState(false);
  return (
    <nav
      className={clsx(
        'shrink-0 border-r border-surface-overlay/40 bg-surface-base/60 ba
        collapsed ? 'w-[64px]' : 'w-[240px]',
      )}
      aria-label="Primary navigation"
    >
      <div className="h-14 flex items-center px-4 border-b border-surface-
        <span className="font-display text-ink-primary text-lg">PSScript</
      </div>
      <ul className="py-3">
        {NAV.map((n) => (
          <li key={n.to}>
            <NavLink
```

```
              to={n.to}
              className={({ isActive }) =>
                clsx(
                  'relative flex items-center h-10 px-4 text-sm text-ink-s
                  isActive && 'text-ink-primary before:absolute before:lef
                )
              }
            >
              {!collapsed ? n.label : n.label[0]}
            </NavLink>
          </li>
        ))}
      </ul>
      <button
        type="button"
        onClick={() => setCollapsed((v) => !v)}
        className="absolute bottom-4 left-4 text-xs text-ink-tertiary hove
        aria-pressed={collapsed}
      >
        {collapsed ? '›' : '‹ collapse'}
      </button>
    </nav>
  );
};
```

- [ ] **Step 5: Implement Topbar**

Create `src/frontend/src/components/layout/Topbar.tsx`:

```
export const Topbar = () => (
  <header className="h-14 flex items-center justify-between px-6 border-b
    <div className="text-xs text-ink-tertiary tabular-nums" aria-live="pol
    <div className="flex items-center gap-3">
      <button
        type="button"
        className="text-xs text-ink-tertiary hover:text-ink-primary border
        aria-label="Open command palette"
      >
        ⌘K
      </button>
    </div>
  </header>
);
```

- [ ] **Step 6: Implement RightRail (slot wrapper)**

Create `src/frontend/src/components/layout/RightRail.tsx`:

```
import { ReactNode } from 'react';

export const RightRail = ({ children }: { children: ReactNode }) => (
  <div className="sticky top-6 space-y-6 text-sm text-ink-secondary">{chil
);
```

- [ ] **Step 7: Layout barrel**

Create `src/frontend/src/components/layout/index.ts`:

```
export { BrandShell } from './BrandShell';
export { OperatorShell } from './OperatorShell';
export { Sidebar } from './Sidebar';
export { Topbar } from './Topbar';
export { RightRail } from './RightRail';
export type { OperatorShellProps } from './OperatorShell';
```

- [ ] **Step 8: Confirm OperatorShell test passes**

```
npm run test:run -- src/components/layout/__tests__/OperatorShell.test.tsx
```

Expected: 3 / 3 pass.

Task E3: Wire shells into App.tsx routing (minimal — Plan 2 finishes the per-page reflow)

Files: `src/frontend/src/App.tsx` (modified)

The goal of this task is **only** to make the shells available in the route tree without yet reflowing pages. Plan 2 will wrap individual routes in `<BrandShell>` / `<OperatorShell>` per pattern; here we just import them and replace the legacy `Layout` component used by the authenticated route group with `OperatorShell` to confirm the wiring works.

- [ ] **Step 1: Find the legacy `Layout` usage**

```
grep -n "Layout" src/App.tsx | head -10
```

Note any line numbers where `<Layout>` wraps a `<Routes>` or route group.

- [ ] **Step 2: Add new shell imports near the top of App.tsx**

Open `src/frontend/src/App.tsx`. Below the existing imports, add:

```
import { BrandShell, OperatorShell } from './components/layout';
```

- [ ] **Step 3: Replace `<Layout>` wrapper with `<OperatorShell>`**

Locate the `<Layout>...</Layout>` JSX that wraps the protected route group. Replace `Layout` with `OperatorShell` in both the opening and closing tags. Leave brand routes (Login, Register, AuthCallback, PendingApproval, NotFound, LandingPage) as they are — they'll be wrapped in `<BrandShell>` per-page in Plan 2 task by task.

If the legacy Layout pulled in `Sidebar` / `Navbar` separately, **leave the legacy Sidebar/Navbar imports for now** — `OperatorShell` renders its own. Plan 2 task L1 will delete the legacy imports.

- [ ] **Step 4: Build and run the dev server briefly**

```
npm run typecheck
npm run build
nohup npm run preview -- --port 4173 --host 127.0.0.1 > /tmp/preview-shell
echo $! > /tmp/preview-shells.pid
sleep 4
curl -fsS http://127.0.0.1:4173/dashboard -o /dev/null && echo OK_DASHBOAR
curl -fsS http://127.0.0.1:4173/login -o /dev/null && echo OK_LOGIN || ech
kill $(cat /tmp/preview-shells.pid)
rm -f /tmp/preview-shells.pid
```

Expected output: `OK_DASHBOARD` and `OK_LOGIN`. The dashboard now renders inside `OperatorShell` (it'll look "off" because the page itself is still legacy, but the shell is correct). Login is unaffected — Plan 2 wraps it in `BrandShell`.

Task E4: Run the full Vitest suite

- [ ] **Step 1: Run everything**

```
npm run test:run
```

Expected output: prior suite count + new Plan 1 tests, all green. Note the count and durations for the PR description.

Task E5: Commit shells

- [ ] **Step 1: Stage and commit**

```
 git add src/frontend/src/components/layout \
        src/frontend/src/App.tsx
git commit -m "$(cat <<'EOF'
feat(redesign): introduce BrandShell + OperatorShell

Wraps the route tree in two new shells: BrandShell sets
[data-surface=brand] and lazy-loads DM Serif Display, with the
GradientField rendering the animated Aurora-glow background;
OperatorShell sets [data-surface=operator][data-theme=dark|light],
mounts Sidebar + Topbar, and exposes an optional rightRail slot
consumed by Pattern 4 (ScriptDetail / ScriptAnalysis) in Plan 2.

App.tsx swaps the legacy <Layout> wrapper for <OperatorShell> across
the protected route group. Brand routes stay unwrapped here — Plan 2
wraps each one in <BrandShell> as part of the per-page reflow.

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
EOF
)"
```

Self-Review

Run this checklist before handing off to execution.

Spec coverage

Spec section	Plan 1 task(s)	Notes
§4 architecture (3 layers)	Phases B, D, E	Tokens → primitives → shells
§5 design tokens (full palette + soft light)	B1–B7	Brand + operator dark + operator light all in tokens.css
§5.6 killed conventions (legacy SaaS blue, fuchsia)	B5	Tailwind colors object replaced wholesale
§5.7 Tailwind shim	B5	colors/boxShadow/borderRadius all read var(--*)
§6 typography (DM Serif, Mona Sans, JetBrains Mono)	C1–C5	Self-hosted via @fontsource; DM Serif lazy in BrandShell
§6.5 tabular numerals	C3	`.tabular-nums` utility defined; Plan 2 applies it
§6.6 README colophon diff	**Plan 3, commit 14**	Out of scope for Plan 1
§7 motion (principles + per-interaction spec)	C4	motion.css with keyframes + reduced-motion enforcement
§7.6 Aurora-glow gradient	D11 (GradientField), E1 (BrandShell)	Component + integration
§8 page patterns	**Plan 2**	Plan 1 only ships shells
§9 verification (route-sweep baseline)	A1–A5	Baseline captured pre-redesign
§9.4 axe-core gate	**Plan 3, commit 12**	Plan 1 doesn't gate yet
§9.5 manual a11y on 4 critical pages	**Plan 3, commit 15**	Plan 1 doesn't manual-pass
§10 shipping plan (commits 1–5)	All of Plan 1	Commits 1, 2, 3, 4, 5 mapped
§10.5 rollback	All commits	Each commit independently revertable

Gaps surfaced for Plan 2: None of the 36 routes are reflowed yet. Plan 2 will pick them up.
Gaps surfaced for Plan 3: axe-core integration, manual a11y pass, README colophon update, visual diff sweep, motion + reduced-motion verification on shipped routes.

Placeholder scan

- No "TBD", "TODO", "implement later" anywhere — every step has working code or an exact command.
- "Add appropriate error handling" / "handle edge cases" — none.
- All test file scaffolds are complete code.
- All file paths are absolute or worktree-relative.

Type / API consistency

- `Button` exposes `variant: 'primary'|'secondary'|'ghost'|'danger'` — same set referenced by tests and shell usages.
- `Card` exposes `density: 'dense'|'comfortable'|'roomy'` — Plan 2 page reflow specs match.
- `Tabs` items have `id, label, disabled?` — matches Plan 2 ScriptAnalysis criteria-tab usage.
- `OperatorShellProps.rightRail` is `ReactNode` — matches Pattern 4 usage in Plan 2.
- Token CSS variable names (`--surface-base`, `--accent`, etc.) match the Tailwind shim's `var(--*)` references.

Scope check

Plan 1 produces working, testable software on its own: - A redesigned token system that any consumer can read. - 11 working primitives with TDD coverage. - Two working shells wrapping the route tree. - A captured visual baseline that Plan 3 will diff against.

The app will look "half-redesigned" between Plans 1 and 2 — the shells will be modern but the inner pages still reference legacy classes that no longer resolve. That's expected and called out in the spec's risk table.

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-04-28-frontend-modernization-plan-1-foundation.md`. Two execution options:

1. **Subagent-Driven (recommended)** — Dispatch a fresh subagent per phase (or per task for the trickier phases B and D). Each subagent reviews the plan, executes its assigned scope, and reports back. The two-stage review pattern from `superpowers:subagent-driven-development` keeps quality high without requiring you to babysit each commit.
2. **Inline Execution** — Execute tasks in this session using `superpowers:executing-plans` with batch execution and checkpoints between phases. Slower per-task but you see everything happen.

Once Plan 1 is complete and merged into the worktree branch, **Plan 2 — Page Reflow** is generated next via `superpowers:writing-plans` against the same spec, picking up where this plan left off (commits 6–10).

Which approach?

Generated 2026-04-29 14:08 UTC